

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №2

по дисциплине: "Логика и основы алгоритмизации в инженерных задачах"

на тему: "Оценка времени выполнения программ"

Выполнил:

Студент группы 24ВВВ3

Мингалёв М.А.

Принял:

к. т. н., доцент, Юрова О. В.

к.т.н., Деев М.В.

Пенза 2025

Цель

Изучение оценки времени выполнения программы

Лабораторное задание

Задание 1:

- Вычислить порядок сложности программы (O-символику).
- Оценить время выполнения программы и кода, выполняющего перемножение матриц, используя функции библиотеки `time.h` для матриц размерами от 100, 200, 400, 1000, 2000, 4000, 10000.
- Построить график зависимости времени выполнения программы от размера матриц и сравнить полученный результат с теоретической оценкой.

Задание 2:

- Оценить время работы каждого из реализованных алгоритмов на случайном наборе значений массива.
- Оценить время работы каждого из реализованных алгоритмов на массиве, представляющем собой возрастающую последовательность чисел.
- Оценить время работы каждого из реализованных алгоритмов на массиве, представляющем собой убывающую последовательность чисел.
- Оценить время работы каждого из реализованных алгоритмов на массиве, одна половина которого представляет собой возрастающую последовательность чисел, а вторая, – убывающую.
- Оценить время работы стандартной функции `qsort`, реализующей алгоритм быстрой сортировки на выше указанных наборах данных.

Результат программы

```
Консоль отладки Microsoft Visual Studio
Размер матрицы | Время умножения (сек) |
-----
Обрабатывается размер 100...
100 | 0,002000 |
Завершено: 100x100 - время: 0,002000 сек.
Обрабатывается размер 200...
200 | 0,011000 |
Завершено: 200x200 - время: 0,011000 сек.
Обрабатывается размер 400...
400 | 0,095000 |
Завершено: 400x400 - время: 0,095000 сек.
Обрабатывается размер 1000...
1000 | 1,803000 |
Завершено: 1000x1000 - время: 1,803000 сек.
Обрабатывается размер 2000...
2000 | 23,858000 |
Завершено: 2000x2000 - время: 23,858000 сек.
Обрабатывается размер 4000...
4000 | 256,364000 |
Завершено: 4000x4000 - время: 256,364000 сек.
Обрабатывается размер 10000...
10000 | 3992,387000 |
Завершено: 10000x10000 - время: 3992,387000 сек.
```

Размер матрицы	Время (сек)
100	0.001
200	0.010
400	0.084
1000	1.582
2000	20.246
4000	261.631
10000	4005.521

Таблица 1



Тип массива	Сортировка Шелл	Быстрая сортировка	Стандартная сортировка
Случайная	0.392000	0.008000	0.013000
Возрастающая	0.001000	0.001000	0.006000
Убывающая	0.772000	0.001000	0.006000
Смешанная	0.384000	1.109000	0.023000

Таблица 2

```

Консоль отладки Microsoft Visual Studio

=== ОЦЕНКА ВРЕМЕНИ РАБОТЫ АЛГОРИТМОВ СОРТИРОВКИ ===
Размер массива: 100000 элементов

-----
| Тип массива | Сортировка Шелла | Быстрая сортировка | Стандартная qsort |
-----
| Случайная | 0,392000 | 0,008000 | 0,013000 |
| Возрастающая | 0,001000 | 0,001000 | 0,006000 |
| Убывающая | 0,772000 | 0,001000 | 0,006000 |
| Смешанная | 0,384000 | 1,109000 | 0,023000 |
-----

=== РЕЗУЛЬТАТЫ ЗАПИСАНЫ В ФАЙЛ sorting_times.txt ===

E:\gitnew\laba2\laba2.2\x64\Debug\laba2.2.exe (процесс 12632) завершил работу с кодом 0 (0x0).
Нажмите любую клавишу, чтобы закрыть это окно:

```

Листинг

Файл 1-laba2.c

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int** create_matrix(int n) {
    int** matrix = (int**)malloc(n * sizeof(int*));
    for (inti = 0; i < n; i++) {
        matrix[i] = (int*)malloc(n * sizeof(int));
    }
    return matrix;
}

void free_matrix(int** matrix, int n) {
    for (inti = 0; i < n; i++) {
        free(matrix[i]);
    }
    free(matrix);
}

void fill_matrix(int** matrix, int n) {
    for (inti = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            matrix[i][j] = rand() % 100 + 1;
        }
    }
}

double multiply_matrices(int n) {
    int** a = create_matrix(n);
    int** b = create_matrix(n);
    int** c = create_matrix(n);

    fill_matrix(a, n);
    fill_matrix(b, n);

    clock_t start = clock();

    for (inti = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            c[i][j] = 0;
            for (int k = 0; k < n; k++) {
                c[i][j] += a[i][k] * b[k][j];
            }
        }
    }

    clock_t end = clock();
    double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;

    free_matrix(a, n);
    free_matrix(b, n);
    free_matrix(c, n);

    return time_taken;
}

int main() {
    int sizes[] = { 100, 200, 400, 1000, 2000, 4000, 10000 };
    int num_sizes = sizeof(sizes) / sizeof(sizes[0]);

    FILE* file = fopen("matrix_multiply_times.txt", "w");
```

```

    if (file == NULL) {
printf("Ошибкаоткрытияфайла!\n");
return 1;
    }

fprintf(file, "Размерматрицы\tВремя (сек)\n");

srand(time(NULL));

    for (inti = 0; i<num_sizes; i++) {
        int n = sizes[i];
printf("Вычислениедляматрицы %dx%d...\n", n, n);

double time_taken = multiply_matrices(n);

fprintf(file, "%d\t%.6f\n", n, time_taken);

printf("Завершено: %dx%d - время: %.6f сек.\n", n, n, time_taken);
    }

fclose(file);
printf("Результатызаписанывфайл matrix_multiply_times.txt\n");

return 0;
}

```

Файл 1-laba2.2.c

```

#define_CRT_SECURE_NO_WARNINGS

#include<stdio.h>
#include<stdlib.h>
#include<time.h>
#include<string.h>
#include<locale.h>

void shell(int* items, intcount) {
    inti, j, gap, k;
    int x, a[5];
    a[0] = 9; a[1] = 5; a[2] = 3; a[3] = 2; a[4] = 1;

    for (k = 0; k < 5; k++) {
        gap = a[k];
        for (i = gap; i<count; ++i) {
            x = items[i];
            for (j = i - gap; (x <items[j]) && (j >= 0); j = j - gap)
                items[j + gap] = items[j];
            items[j + gap] = x;
        }
    }
}

voidqs(int* items, intleft, intright) {
    if (left>= right) {
        return;
    }

    while (left<right) {
        inti = left;
        int j = right;
        int pivot = items[(left + right) / 2];

        while (i<= j) {
            while (items[i] < pivot) i++;
            while (items[j] > pivot) j--;
            if (i<= j) {

```

```

int temp = items[i];
items[i] = items[j];
items[j] = temp;
i++;
j--;
    }
}

if (j - left < right - i) {
qs(items, left, j);
left = i;
}
else {
qs(items, i, right);
right = j;
}
}

int compare(constvoid* a, constvoid* b) {
return (*(int*)a - *(int*)b);
}

void generate_random(int* arr, int n) {
for (int i = 0; i < n; i++) {
arr[i] = rand() % 10000;
}
}

void generate_ascending(int* arr, int n) {
for (int i = 0; i < n; i++) {
arr[i] = i;
}
}

void generate_descending(int* arr, int n) {
for (int i = 0; i < n; i++) {
arr[i] = n - i;
}
}

void generate_mixed(int* arr, int n) {
for (int i = 0; i < n / 2; i++) {
arr[i] = i;
}
for (int i = n / 2; i < n; i++) {
arr[i] = n - i;
}
}

int* copy_array(const int* source, int n) {
int* copy = (int*)malloc(n * sizeof(int));
if (copy == NULL) {
printf("Ошибка выделения памяти!\n");
exit(1);
}
memcpy(copy, source, n * sizeof(int));
return copy;
}

double measure_shell_time(int* arr, int n) {
int* temp = copy_array(arr, n);
clock_t start = clock();
shell(temp, n);
clock_t end = clock();
free(temp);
}

```

```

return ((double)(end - start)) / CLOCKS_PER_SEC;
}

double measure_qs_time(int* arr, int n) {
int* temp = copy_array(arr, n);
clock_t start = clock();
qs(temp, 0, n - 1);
clock_t end = clock();
free(temp);
return ((double)(end - start)) / CLOCKS_PER_SEC;
}

double measure_std_qsort_time(int* arr, int n) {
int* temp = copy_array(arr, n);
clock_t start = clock();
qsort(temp, n, sizeof(int), compare);
clock_t end = clock();
free(temp);
return ((double)(end - start)) / CLOCKS_PER_SEC;
}

int main() {
setlocale(LC_ALL, "rus");
const int n = 100000;
int* original_array = (int*)malloc(n * sizeof(int));

if (original_array == NULL) {
printf("Ошибка выделения памяти!\n");
return 1;
}

const char* array_types[] = {
"Случайная",
"Возрастающая",
"Убывающая",
"Смешанная"
};

void (*generators[])(int*, int) = {
generate_random,
generate_ascending,
generate_descending,
generate_mixed
};

FILE* file = fopen("sorting_times.txt", "w");
if (file == NULL) {
printf("Ошибка открытия файла!\n");
free(original_array);
return 1;
}

printf("=== ОЦЕНКА ВРЕМЕНИ РАБОТЫ АЛГОРИТМОВ СОРТИРОВКИ ===\n");
printf("Размер массива: %d элементов\n", n);

printf("-----\n");
printf("| %-15s | %-20s | %-20s | %-15s |\n",
"Тип массива", "Сортировка Шелла", "Быстрая сортировка", "Стандартная qsort");
printf("-----\n");

fprintf(file, "=== ОЦЕНКА ВРЕМЕНИ РАБОТЫ АЛГОРИТМОВ СОРТИРОВКИ ===\n");
fprintf(file, "Размер массива: %d элементов\n", n);
fprintf(file, "-----\n");
fprintf(file, "| %-15s | %-20s | %-20s | %-15s |\n",
"Тип массива", "Сортировка Шелла", "Быстрая сортировка", "Стандартная qsort");
fprintf(file, "-----\n");

```



```
srand((unsignedint)time(NULL));

for (inti = 0; i< 4; i++) {
    generators[i](original_array, n);

    doubletime_shell = measure_shell_time(original_array, n);
    doubletime_qs = measure_qs_time(original_array, n);
    doubletime_std_qsort = measure_std_qsort_time(original_array, n);

    printf("| %-15s | %-20.6f | %-20.6f | %-15.6f | \n",
        array_types[i], time_shell, time_qs, time_std_qsort);

    fprintf(file, "| %-15s | %-20.6f | %-20.6f | %-15.6f | \n",
        array_types[i], time_shell, time_qs, time_std_qsort);
}

printf("-----\n");
fprintf(file, "-----\n");

fclose(file);
free(original_array);

printf("\n=== РЕЗУЛЬТАТЫ ЗАПИСАНЫ В ФАЙЛ sorting_times.txt ===\n");

return 0;
}
```

Вывод

В ходе выполнения лабораторной работы были разработаны программы для выполнения заданий Лабораторной работы №2. В процессе выполнения работы были использованы знания о оценке времени.

Ссылка на удаленный репозиторий GitHub:

<https://github.com/MakarMn/LiOAvIZ>